

3D Fluid Sim

Maanav Koladia, Andres Wearden, Shrikar Thangvel, Kareem Mohamed

The University of Texas at Austin

{mak4738,aeq3364,km52839,st36352}@eid.utexas.edu

Abstract

We are building a real-time 3D fluid simulation that produces visually realistic water using CUDA for physics and OpenGL for rendering. Our goal is to have a live demo of the sim at the end of the semester and gain hands-on experience multithreaded programming during the development process

1 Introduction

Can we make a grid-based fluid simulation that runs smoothly using GPUs, where the water doesn't squish or expand and the total amount of water stays relatively the same over a long run?

2 Background

A grid-based fluid simulation is important because they have broad applications across the industry and entertainment. For example, real-time graphics and games have believable water that doesn't appear to compress, expand, or slowly vanish which is essential for quality of the game. Another example is in the film industry, where this can save artists time by reducing artifacts that would otherwise need manual correction. Engineering fields also rely on fast, grid-based incompressible fluid solvers for tasks such as HVAC design, flood prediction, maritime engineering, and digital twins, where interactive or low-cost pre-visualization is valuable before high-fidelity simulations. Robotics, medical training, and AR/VR systems benefit as well, since real-time mass-conserving simulations help with accurate interaction feedback in dynamic environments. Ultimately, advancing grid-based fluid simulations ensures more realistic, efficient, and reliable tools that enhance creativity, improve decision-making, and expand what's possible across both entertainment and engineering applications.

3 Approach

We will implement a real-time 3D fluid simulation using CUDA for physics and OpenGL for rendering. CUDA kernels will compute advection, pressure projection, and free-surface tracking, while OpenGL will handle rendering and shading. The CPU will coordinate simulation steps and input.

3.1 Resources Needed

- **Hardware:** NVIDIA CUDA-capable GPU, multi-core CPU.
- **Software:** CUDA toolkit, OpenGL, Nsight.

3.2 Uncertainties

- Pressure solver performance (Jacobi vs. Conjugate Gradient/Multigrid)
- Surface method (marching cubes vs. raymarching)
- CUDA optimization (memory layout, block/grid configuration)
- Balancing realism with visual quality

3.3 Task Division (4 People)

- **Maanav:** Velocity field and scalar advection kernels
- **Andres:** Pressure solver, projection, divergence computation
- **Shrikar:** Free-surface (level set) tracking, surface extraction
- **Kareem:** Rendering pipeline, water shaders, UI, and demo integration

All members will collaborate on debugging, performance optimization, and final demo preparation.

4 Evaluation

We plan to evaluate the idea by defining clear and measurable criteria that align with our project goals, such as accuracy, efficiency, usability, and scalability. The prototype will be tested against baseline methods or existing solutions to determine whether it offers meaningful improvements. We will also perform stress testing under different scenarios to assess robustness and reliability. Where applicable, we will gather feedback from peers, mentors, or potential users to gain insights into practicality and ease of use. By combining quantitative performance metrics with qualitative feedback, we can comprehensively evaluate both the technical effectiveness and the real-world applicability of our idea.

5 Related Work

None right now

6 Conclusion

Timeline of the project, using deadlines from the project description.

1. September 5th – Project groups must be set.
2. September 19th – Proposal due.
3. October 15th – Design Document due: we will clearly outline the technical implementation plan from start to finish, and detail potential hurdles with strategies for addressing them.

4. October 31st – Implementation and Demo Checkpoint I: display API completed and a working 2D fluid simulation.
5. November 30th – Implementation and Demo Checkpoint II: full 3D simulation implemented.
6. December 2nd and December 4th – Project presentations in class: present the full 3D simulation, demonstrate user interaction, explain key implementation details, and answer questions.
7. December 5th – Final project report due.